Arabic Headline Generation

Course Code: AIN4101

Course Name: Machine Learning II

Title: Arabic News Headline Generation with SimpleRNN and LSTM Encoder-Decoder Models

Doctor Name: *Badrya Dahy*

Supervisor: *Sarah Ahmed*

Project Team:

Name	ID
Ali Omar	2023025430
Zizi Mostafa	2023019652
Shaza Abdulnasser	2023017083

Contents

Introduction..... 2

Method..... 3

Dataset Overview and Preparation..... 4

Experiment..... 6

Results..... 7

Discussion..... 9

Conclusion..... 10

Future Work..... 10

Abstraction

This project presents an Arabic news headline generation system built with sequence-to-sequence deep learning. Given a financial news article, an encoder-decoder network writes a short headline for it. Two recurrent architectures, a SimpleRNN and an LSTM, were trained on a 5,000-article sample of the AraFinNews dataset, tuned on 1,000 validation articles and tested on 1,000 unseen articles. Quality was measured with BLEU-1 and ROUGE-L. The LSTM scored higher (BLEU-1 0.0736, ROUGE-L 0.0779) than the SimpleRNN (0.0587 and 0.0615), but both scores are low and the generated headlines are generic rather than faithful to the article. The results show how strongly training-set size, vocabulary size and the lack of an attention mechanism limit word-level encoder-decoder models. The final system is published as a Streamlit web application and an open GitHub repository.

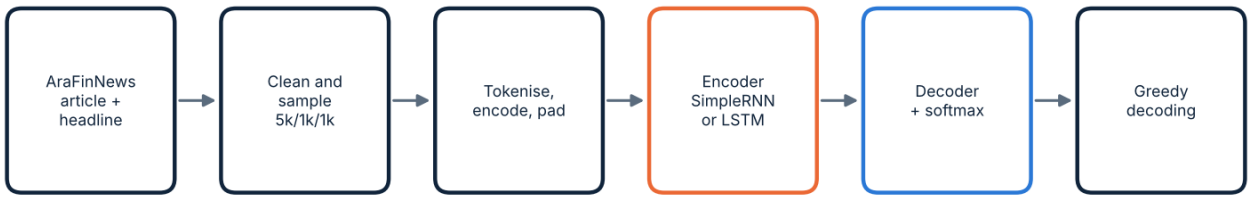
Introduction

Writing a headline is a form of abstractive summarisation: the headline must capture the main fact of an article using words that may not appear in it. Newsrooms, financial platforms and search engines produce thousands of such headlines every day, so automatic headline generation is a practical natural language processing task. Arabic adds extra difficulty because of its rich morphology, optional diacritics, several spellings of the same letter, and words that carry attached prefixes and pronouns.

This project builds and compares two classic neural approaches to the task, both using the encoder-decoder (sequence-to-sequence) design. The encoder reads the first words of an article and compresses them into a state vector; the decoder starts from that state and writes the headline one word at a time. The only difference between the two experiments is the recurrent layer: a SimpleRNN in the baseline and an LSTM in the second model. The models are trained on Arabic financial news from the AraFinNews corpus, evaluated with BLEU-1 and ROUGE-L, and deployed as an interactive Streamlit application so that the outputs can be inspected side by side.

Method

The system follows the standard encoder-decoder framework for text generation. An article is cleaned, converted into integer word ids and passed through an encoder. The encoder's final state initialises a decoder, which is trained with teacher forcing: at every step it receives the correct previous headline word and learns to predict the next one. At prediction time the decoder starts from a start token [SOS] and feeds its own output back in until it produces an end token [EOS] or reaches the length limit (greedy decoding).



Teacher forcing during training; the encoder's final state initialises the decoder

Two main experiments were conducted using different recurrent layers:

- **Experiment 1, baseline:** a SimpleRNN encoder-decoder with 128-dimensional word embeddings and 128 recurrent units.
- **Experiment 2, improved model:** the same network with LSTM layers. The LSTM has gates and a separate cell state, so the encoder passes both a hidden state and a cell state to the decoder. This is intended to retain information over longer sequences.

Both models share the same data, vocabulary, hyper-parameters and random seed (42), so any difference in results comes from the recurrent layer. The models were trained on a training set of 5,000 article-headline pairs, monitored on a separate validation set, and evaluated with BLEU-1 and ROUGE-L on a test set that was never used during training or tuning.

Dataset Overview and Preparation

1. Dataset Selection

The project uses **AraFinNews**, a large collection of Arabic financial news articles, each paired with its published headline (Hugging Face: drelhaj/AraFinNews). The notebook reads the parquet version of the dataset, which has a training split of **340,024** rows, a validation split of **42,502** rows and a test split of **42,504** rows.

Exactly half of the rows in every split are empty (170,012 in train, 21,251 in validation and 21,252 in test have no article and no headline), so roughly 170,000 usable training pairs remain. Because each training epoch took about 70 seconds on the available hardware, a reproducible random **subset** was used: **5,000** pairs for training, **1,000** for validation and **1,000** for testing.

Subset statistics

Subset	Pairs	Avg. article words	Avg. headline words
Training	5,000	181.57	10.32
Validation	1,000	182.45	10.25
Test	1,000	193.70	10.29

Word counts are measured on the raw text before cleaning. Headlines are about 10 words long on average, so a short decoder limit of 19 steps is enough.

2. Challenges Faced During Data Preparation

The following issues came from the nature of Arabic news text and from the limits of the available compute, not from errors in the dataset itself.

- **Empty rows:** half of every split had missing text and had to be removed before sampling.
- **Duplicates and markup:** a few duplicate pairs and HTML entities such as " appeared in headlines; both were cleaned.
- **Spelling variants:** the same word can be written with different alef forms or with diacritics and tatweel. These were normalised, although other variants (such as ي and ى, or ة and ه) were left unchanged and still split the vocabulary.
- **Punctuation attached to words:** tokens are separated only by spaces, so a word followed by a full stop or comma becomes a different token from the same word on its own. This adds many rare entries to the vocabulary.
- **Very large vocabulary for a small sample:** 92,121 distinct tokens for only 5,000 training articles means most words occur once or twice, which makes them hard to learn.
- **Long articles, short context:** articles average about 182 to 194 words, but only the first 80 are given to the encoder, so the end of the article is never seen.
- **Computing time:** about 70 seconds per epoch (23 to 24 minutes per model) limited the number of training pairs and the amount of tuning that was practical.

3. Data Preparation Pipeline

Step	What is done
Remove invalid rows	Drop rows with missing or blank article or headline, then drop duplicate pairs (23 in train, 1 in validation, 0 in test).
Clean Arabic text	Convert HTML entities (" to a quotation mark), remove URLs and HTML tags, remove diacritics and tatweel, unify the alef forms (أ إ آ ٱ to ا), collapse spaces.

Sample	Shuffle with seed 42 and keep 5,000 / 1,000 / 1,000 pairs; an article selected for one split cannot appear in another.
Tokenise and build vocabulary	Split on spaces. Vocabulary built from training articles and headlines: 92,121 entries including [PAD]=0, [UNK]=1, [SOS]=2, [EOS]=3. Unknown words map to [UNK].
Encode and pad	Articles are cut to 80 tokens. Headlines are cut to 18 words, wrapped in [SOS]/[EOS] and padded to 20. The decoder input is the first 19 tokens and the target is the same sequence shifted by one.

Dataset direct link: https://huggingface.co/datasets/drelhaj/AraFinNews

Experiment

Experiment Setup

Two training experiments were run with the same data pipeline to measure the effect of the recurrent layer type on headline quality.

Experiment 1 (SimpleRNN)

In this experiment the encoder and decoder each use a SimpleRNN layer with the following configuration:

- Embedding size: 128 (separate embeddings for encoder and decoder, vocabulary 92,121)
- Recurrent units: 128, dropout 0.1
- Output layer: dense softmax over the whole vocabulary
- Trainable parameters: about 35.5 million (more than 99% are in the embeddings and output layer)
- Training time: about 23 minutes (20 epochs)

This setup was used as the baseline.

Experiment 2 (LSTM)

In this experiment the same architecture was used with LSTM layers, and the encoder's hidden state and cell state both initialise the decoder:

- Embedding size: 128, recurrent units: 128, dropout 0.1
- Trainable parameters: about 35.7 million
- Training time: about 24 minutes (20 epochs)

Shared Training Settings

- Optimizer: Adam, learning rate 0.0005, gradient clipping (clipnorm 1.0)
- Batch size: 32, epochs: 20, random seed: 42
- Loss: sparse categorical cross-entropy; padding positions are excluded with sample weights
- Decoding: greedy (most probable word at each step), maximum 19 words
- Evaluation: BLEU-1 and a word-level ROUGE-L (F-score of the longest common subsequence), computed on whitespace tokens of the cleaned headline

Deployment

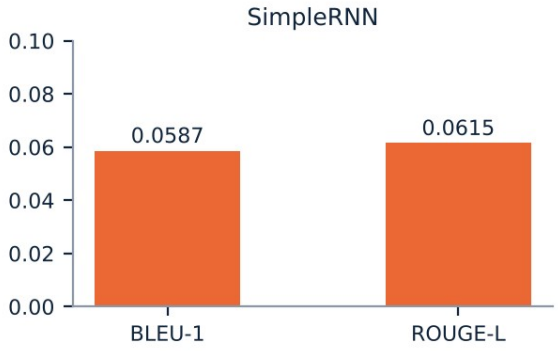
The notebook models are served in a Streamlit application (app.py in the repository). The app is deployed on Streamlit Community Cloud at https://arabic-headline-generation.streamlit.app/. It generates a headline from each model for any pasted Arabic article, using the trained weights stored in the repository (artifacts folder), and shows the test-set scores and training curves. Inference is implemented in NumPy, so the deployed app does not need TensorFlow. For the ten recorded test articles, the app reproduces all 20 notebook headlines (10 per model) exactly.

Results

Both models were evaluated on the same test set of 1,000 unseen articles using **BLEU-1** and **ROUGE-L**, so the comparison is fair. A perfect score on either metric would be 1.0.

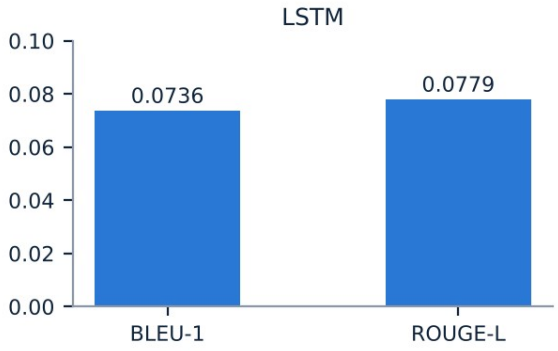
SimpleRNN Results (Baseline Model)

- BLEU-1: **0.0587**
- ROUGE-L: **0.0615**
- Best validation loss: **6.98** (epoch 10)
- Final accuracy: train **23.2%**, validation **16.4%**



LSTM Results (Improved Model)

- BLEU-1: **0.0736**
- ROUGE-L: **0.0779**
- Best validation loss: **7.17** (epoch 19)
- Final accuracy: train **19.4%**, validation **13.1%**



Performance Comparison

- **LSTM achieved higher BLEU-1 and ROUGE-L**: +0.0150 and +0.0163, a relative gain of about 25% and 27%. Its headlines share more words with the reference.
- **SimpleRNN achieved lower validation loss and higher token accuracy** (7.11 vs 7.17 and 16.4% vs 13.1% at epoch 20), so it fits individual next-word predictions slightly better.
- Both models score below 0.08 on both metrics. With one training run and 1,000 test articles, the gap between them should be read as an indication, not as a proven difference.

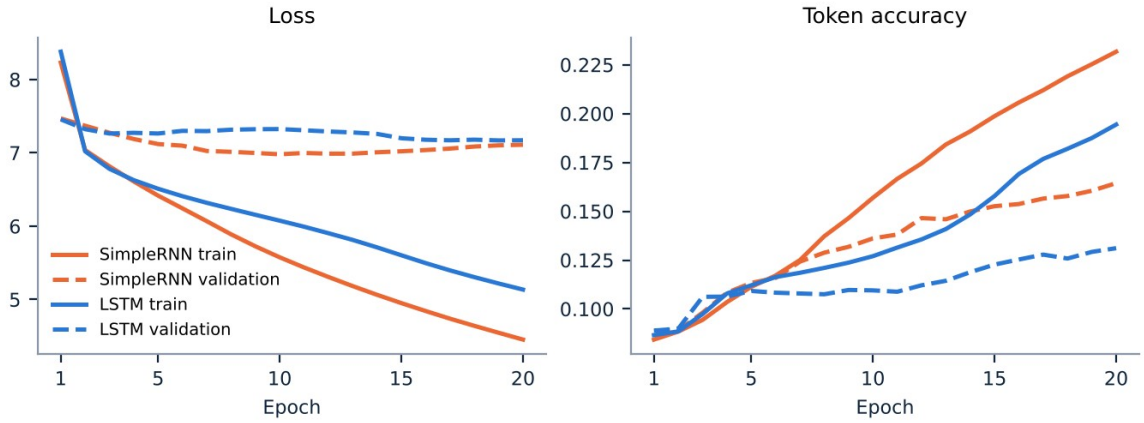


Figure: training (solid) and validation (dashed) loss and token accuracy for the 20 epochs. Validation loss of the SimpleRNN rises after epoch 10 while its training loss keeps falling.

Examples from the Test Set

Headlines recorded in the notebook for three test articles (text is normalised: no diacritics, unified alef):

Article 1: Reference	الكرملين: يجب رفع العقوبات لتسليم الحبوب الروسية للاسواق
SimpleRNN	تراجع الاسهم الامريكية في بداية الجلسة مع ارتفاع الاسهم
LSTM	ارتفاع الاسهم الامريكية من ارتفاع في الولايات المتحدة
Article 6: Reference	المكسيك تسجل 881 اصابة جديدة بفيروس كورونا و34 وفاة
SimpleRNN	وكالة: الصين تسجل ارباحا عن النفط الى اتفاق في الصين
LSTM	ارتفاع الاسهم الامريكية من ارتفاع مع النفط
Article 8: Reference	مؤشر "شنغهاي المركب" يغلق على ارتفاع لثالث جلسة على التوالي
SimpleRNN	تقرير: الصين تعتزم ان شركة الشركات من ان ترامب ذاتية القيادة في الصين
LSTM	ارتفاع اسعار النفط الامريكية في الولايات المتحدة في الربع الثالث

Live application: https://arabic-headline-generation.streamlit.app/

Source code and results: https://github.com/salama1229/arabic-headline-generation

Discussion

The results show that, with this amount of data and this architecture, headline generation is far from solved. The two models disagree on which one is better depending on the measure: the LSTM wins on headline-level overlap (BLEU-1, ROUGE-L), while the SimpleRNN wins on validation loss and next-word accuracy. Token accuracy rewards predicting frequent words, so a model can improve it without producing better headlines; BLEU-1 and ROUGE-L are closer to what a reader sees.

Reading the generated headlines explains the low scores. On the first ten test articles, every LSTM headline opens with one of two phrases, either "ارتفاع" (rise) or "وكالة: الصين تعتزم" (agency: China intends to), and eight of the ten SimpleRNN headlines mention China. The headlines are grammatical-looking but unrelated to the article: an article about the Kremlin and sanctions produced "US stocks fall at the start of the session". The models have learned which headline openings are common in the training data, not how to summarise a specific article. Greedy decoding then strengthens this effect, because the single most likely word is chosen at every step, and it also causes loops such as "ارتفاع في ارتفاع".

Several factors probably contribute, and they are consistent with the training curves:

- **Data and vocabulary:** 5,000 training pairs against a 92,121-word vocabulary and about 35 million parameters. Most words are seen once or twice, so their embeddings cannot be learned reliably, and the model memorises the training set: SimpleRNN training accuracy reaches 23% while validation accuracy stays near 16%, and its validation loss starts to rise after epoch 10.
- **No attention mechanism:** the whole article has to be squeezed into a single 128-number state, and only its first 80 words are used, so important details later in the article are lost.
- **Tokenisation:** splitting only on spaces leaves punctuation attached to words and gives every inflected form of a word its own id, which makes the vocabulary sparse.
- **Training signal:** teacher forcing and cross-entropy reward the most frequent next word, which favours generic openings over article-specific content.

The LSTM's slower decrease in training loss (5.13 vs 4.45 at epoch 20) shows it fits the training data less aggressively. This is consistent with its better generalisation on the test metrics, but because only one run per model was made, the project cannot separate this effect from random variation between runs.

Overall, the project met its goal of building, comparing and deploying two encoder-decoder baselines, and it clearly identifies where they fall short.

Conclusion

In this project, an Arabic news headline generation system was developed using sequence-to-sequence deep learning. A SimpleRNN and an LSTM encoder-decoder were trained on a 5,000-article sample of the AraFinNews dataset after a full Arabic preprocessing pipeline, and evaluated on 1,000 unseen articles with BLEU-1 and ROUGE-L. The system was also deployed as a Streamlit web application connected to a public GitHub repository.

The LSTM obtained the better headline-overlap scores (BLEU-1 0.0736, ROUGE-L 0.0779 against 0.0587 and 0.0615 for the SimpleRNN), but both models achieved only low performance. Their headlines are fluent-looking yet generic, and they rarely reflect the content of the article. The experiments highlight that dataset size, vocabulary design, context length and the absence of attention are the main limits of word-level recurrent models, and that automatic metrics should be read together with sample outputs.

Future Work

1. Use More Data and a Better Vocabulary:

Train on the full set of about 170,000 usable pairs, apply a minimum word frequency, and move to subword tokens (SentencePiece or BPE) with Arabic-aware normalisation and punctuation splitting. This directly attacks the sparse 92,121-word vocabulary.

2. Add Attention or Use a Pretrained Arabic Model:

Add an attention layer so that the decoder can look at every article word, or fine-tune a pretrained Arabic sequence-to-sequence model such as AraT5, AraBART or mT5.

3. Improve Decoding:

Replace greedy decoding with beam search plus a repetition penalty and length normalisation to reduce loops and generic openings.

4. Evaluate More Thoroughly:

Report ROUGE-1/2/L and BERTScore, add a simple baseline that uses the first words of the article, run several seeds to measure variance, and include a small human evaluation of relevance and fluency.

5. Extend the Application:

Let users choose the decoding strategy, show the words of the article the model attends to, and collect feedback on generated headlines.